

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК И КИБЕРБЕЗОПАСНОСТИ
ВЫСШАЯ ШКОЛА ПРОГРАММНОЙ ИНЖЕНЕРИИ

Отчет о прохождении производственной практики
на тему: «Разработка ML-пайплайнов обработки и анализа
больших данных
с использованием ClearML и PyTorch»

Михайлова Сергея Никитовича, гр. 5130903/30302

Направление подготовки: 09.03.03 Прикладная информатика.

Место прохождения практики: СПбПУ, ИКНК, ВШ ПИ.

Сроки практики: с 11.06.2026 по 09.07.2026.

Руководитель практики от ФГАОУ ВО «СПбПУ»: Пархоменко
Владимир Андреевич, старший преподаватель.

Оценка: _____

Руководитель практики
от ФГАОУ ВО «СПбПУ»

В. А. Пархоменко

Обучающийся

С. Н. Михайлов

Дата: 09.07.2026

Содержание

1	Введение	3
1.1	Актуальность	3
1.2	Постановка задачи	3
1.3	Задачи	4
1.4	Сценарии использования (Use Cases)	5
2	Анализ предметной области и технологического стека	6
2.1	Исходные данные	6
2.2	Задача ранжирования	7
2.3	Технологический стек	7
2.4	Выбор системы трекинга экспериментов	8
2.5	Развертывание ClearML	9
2.6	Архитектура решения	10
3	Пайплайн подготовки данных	11
3.1	Индексация данных и временной порядок	11
3.2	Очистка данных и целевая переменная	11
3.3	Признаки	12
3.4	Эмбеddинг-профили пользователей	13
3.5	Сборка датасетов	13
4	Обучение ранжирующей модели и оффлайн-метрики	14
4.1	Модель	14
4.2	Метрики на этапе обучения	15
4.3	Собственный модуль оффлайн-метрик	15
4.4	Трекинг экспериментов	15
4.5	Результаты обучения и оценки модели	16
5	Тестирование пайплайна и модели	17
5.1	Подход к тестированию	17
5.2	Уровни тестирования	18
5.3	Матрица покрытия	18
5.4	Запуск тестов	19
	Заключение	19
	Список литературы	22
	Приложение А. Исходный код и конфигурации ключевых компонентов	23

1 Введение

1.1 Актуальность

Ленты коротких видео стали одним из основных форматов потребления контента: сервисы ежедневно обслуживают миллионы пользователей, а количество зарегистрированных взаимодействий (просмотров, лайков, шерингов) измеряется миллиардами событий. В таких условиях качество ленты напрямую определяется системой ранжирования — моделью, которая для каждого пользователя упорядочивает кандидатов-видео по ожидаемой полезности.

Построение промышленной системы ранжирования требует решения двух связанных задач. Во-первых, это задача инженерии данных (Data Engineering): исходные журналы взаимодействий имеют объем в десятки миллионов строк, требуют очистки, агрегации и трансформации в признаковое описание, пригодное для машинного обучения, причем обработка должна выполняться в потоковом режиме, не загружая весь объем в память. Во-вторых, это задача машинного обучения: выбор целевой переменной, отражающей интерес пользователя, построение признаков без утечек будущего, обучение ранжирующей модели и ее корректная оценка оффлайн-метриками ранжирования.

Отдельное требование современной ML-разработки — воспроизводимость экспериментов. Каждая стадия обработки данных и каждое обучение модели должны фиксироваться в системе трекинга экспериментов вместе с конфигурацией, метриками и артефактами. В работе для этого используется платформа ClearML [1], развернутая локально в Docker.

1.2 Постановка задачи

В рамках производственной практики были поставлены следующие задачи:

1. Провести анализ требований к обработке и анализу больших данных для задачи ранжирования коротких видео.
2. Разработать пайплайн сбора, хранения и предобработки данных с использованием инструментов Data Engineering (Polars, PyArrow).
3. Настроить процессы очистки, агрегации и трансформации данных для последующего анализа и применения ML-моделей.
4. Провести анализ данных, рассчитать ключевые метрики, сформировать признаки и подготовить датасеты для машинного обучения.

5. Обучить и оценить ранжирующую модель с использованием ClearML, CatBoost [2] и PyTorch [3]; обучение итоговой модели выполнить на GPU.
6. Протестировать реализованные пайплайны и ML-модели автоматизированными тестами.
7. Подготовить итоговый отчет с описанием архитектуры решения, выбранного стека и результатов.

1.3 Задачи

1. Изучить структуру исходного датасета взаимодействий пользователей с короткими видео: недельные шарды событий, метаданные пользователей и видео, контентные эмбединги.
2. Реализовать этап индексации данных с сортировкой шардов по глобальному временному порядку.
3. Реализовать потоковую очистку событий и построение градуированной целевой переменной (релевантности) из неявного фидбека.
4. Спроектировать и вычислить три группы признаков — пользовательские, айтемные и парные (пользователь–видео), включая признак близости контентного эмбединга видео к эмбединг-профилю пользователя (расчет на PyTorch).
5. Выполнить темпоральное разбиение данных на train/validation/test и собрать датасеты в формате, требуемом ранжирующей моделью CatBoost.
6. Реализовать собственный модуль оффлайн-метрик ранжирования (NDCG@k, MAP@k, MRR@k, HitRate@k, Recall@k, Precision@k, GAUC) в векторизованном виде.
7. Обучить модель **CatBoostRanker** (лосс YetiRank [4]) с расчетом метрик на валидации в процессе обучения и логированием в ClearML.
8. Покрыть пайплайн и модель автоматизированными тестами на синтетических данных.

Исходный код решения вместе с используемыми данными опубликован в открытом репозитории: <https://github.com/seregamikhailov/ml-training-catboost>.

1.4 Сценарии использования (Use Cases)

UML-диаграмма прецедентов разработанного решения приведена на рисунке 1. Актерами выступают инженер (запускает пайплайн и обучение), исследователь (анализирует результаты) и сервер ClearML как внешняя система трекинга.

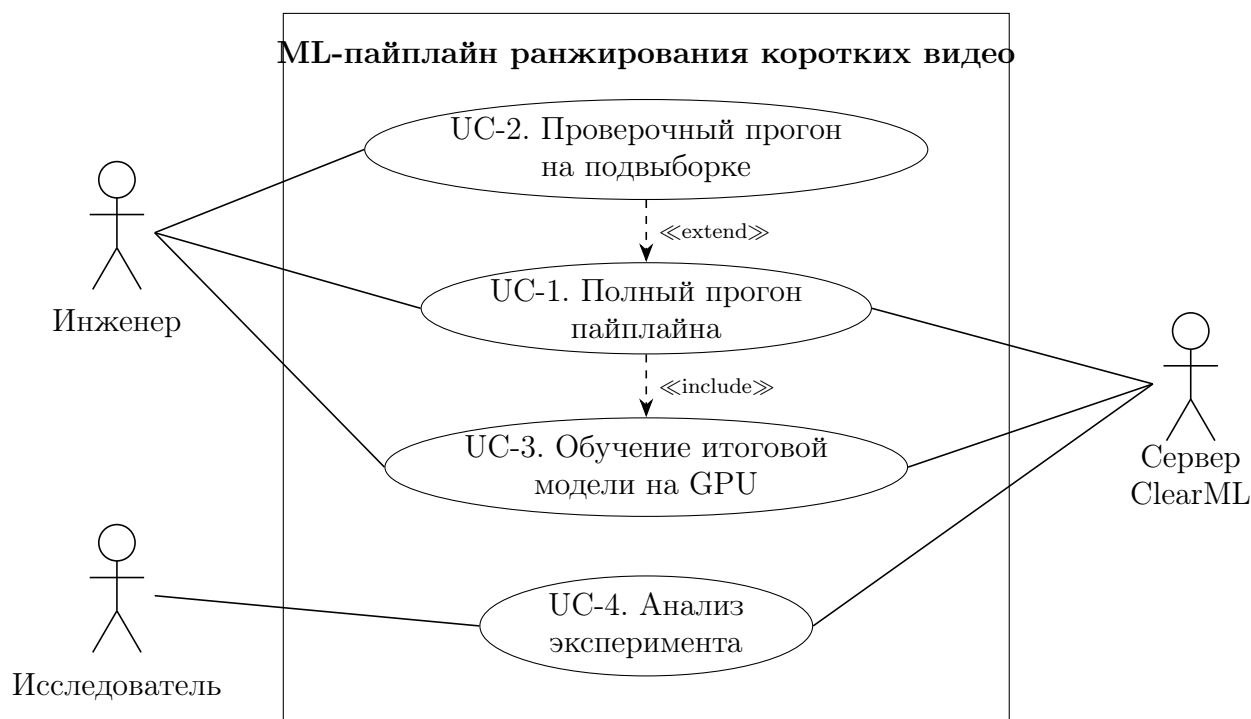


Рис. 1: UML-диаграмма прецедентов разработанного решения

UC-1. Полный прогон пайплайна. Инженер запускает команду `python -m src.pipeline`. Система последовательно выполняет пять этапов: индексацию данных, очистку, построение признаков, сборку датасетов и обучение модели. Каждый этап создает отдельный эксперимент в ClearML с конфигурацией, статистиками и артефактами.

UC-2. Быстрый проверочный прогон. Инженер ограничивает объем данных параметрами конфигурации (`max_shards`, `user_fraction`, `max_rows`) и прогоняет пайплайн на подвыборке за минуты, проверяя работоспособность всех этапов перед полномасштабным запуском.

UC-3. Обучение итоговой модели на GPU. Инженер запускает этап `train` на сервере с GPU по полному конфигу (`task_type: GPU`). На машине без CUDA обучение автоматически продолжается на CPU (механизм `allow_cpu_fallback`), что позволяет использовать один и тот же код локально и на сервере.

UC-4. Анализ эксперимента. Исследователь открывает веб-интерфейс ClearML, сравнивает кривые метрик по итерациям обучения,

изучает важности признаков и финальные оффлайн-метрики на валидации и тесте, скачивает артефакт модели.

2 Анализ предметной области и технологического стека

2.1 Исходные данные

Работа выполняется на открытом датасете взаимодействий пользователей с короткими видео. Полный объем исходных данных составляет около 1.5 ТБ (десятки миллиардов событий, миллионы пользователей и видео) — обработка такого объема на одной машине нецелесообразна, поэтому в соответствии с заданием практики использована репрезентативная часть датасета (≈ 2.6 ГБ). Прореживание выполнено по пользователям и видео с сохранением всех событий отобранных сущностей, поэтому распределения активности и плотность взаимодействий в выборке соответствуют полным данным, а пайплайн спроектирован потоковым и масштабируется на большую долю датасета изменением одного параметра конфигурации.

Используемая выборка охватывает 27 последовательных недель и содержит:

- **47 976 280 событий** взаимодействия (10 000 пользователей, 19 628 видео);
- по каждому событию — время просмотра `timespent`, явный фидбек (`like`, `dislike`, `share`, `bookmark`, `click_on_author`, `open_comments`) и контекст показа (`place`, `platform`, `agent`);
- метаданные пользователей (возраст, пол, регион) и видео (длительность, идентификатор автора);
- контентные эмбединги видео размерности 64 в формате `npz`; в работе используются первые 32 компоненты (Matryoshka-представление).

Ключевое свойство датасета — глобальный временной порядок: события отсортированы по времени как внутри недельных шардов, так и между ними. Это позволяет строить темпоральные разбиения `train/validation/test` и признаки без утечки будущего.

Структура исходных файлов приведена в таблице 1.

Таблица 1: Структура исходных данных

Файлы	Содержимое
**/week_NN.parquet	Недельные шарды взаимодействий: <code>user_id</code> , <code>item_id</code> , <code>timespent</code> , фидбек, контекст
metadata/users_metadata.parquet	Пользователи: <code>age</code> , <code>gender</code> , <code>geo</code>
metadata/items_metadata.parquet	Видео: <code>duration</code> , <code>author_id</code>
metadata/item_embeddings.npz	Массивы <code>item_id</code> и <code>embedding</code> (контентные эмбединги)

Особенность хранения — компактные типы: идентификаторы `UInt32`, время просмотра и длительность `UInt8`, фидбек `Boolean`, категориальные атрибуты — целочисленные коды `UInt8`. Это в разы сокращает объем на диске и в памяти по сравнению со строковыми представлениями.

2.2 Задача ранжирования

Задача формулируется как *learning-to-rank*: для каждого пользователя (группы) модель должна упорядочить показанные ему видео так, чтобы более релевантные оказались выше. В отличие от бинарной классификации, оптимизируется именно порядок внутри группы, а основной метрикой качества служит `NDCG@k`.

Целевая переменная (релевантность) строится из неявного фидбека: просмотр видео и позитивные действия увеличивают релевантность с настраиваемыми весами, дизлайк обнуляет ее. Такой градуированный таргет информативнее бинарного «клик/не клик» и естественно сочетается с *listwise-лосс*ами.

2.3 Технологический стек

Выбранный стек приведен в таблице 2.

Таблица 2: Технологический стек решения

Инструмент	Роль	Обоснование выбора
Polars [5]	обработка данных	ленивые вычисления и потоковый движок: агрегации и джойны на 48М строк без загрузки в память
PyArrow [6]	колоночный I/O	батчевое чтение parquet и инкрементальная запись (ParquetWriter) для расчета эмбединга-признака
PyTorch [3]	векторные вычисления	расчет эмбединга-профилей пользователей и косинусов на GPU/MPS/CPU единым кодом
CatBoost [2]	ранжирующая модель	встроенные listwise-лоссы (YetiRank [4]), нативная поддержка категориальных признаков и пропусков, обучение на GPU
scikit-learn [7]	верификация	эталонная реализация NDCG для сверки собственных метрик в тестах
ClearML [1]	трекинг экспериментов	self-hosted сервер, автоматическая фиксация конфигураций, скаляров и артефактов
Docker Compose	инфраструктура	локальное развертывание сервера ClearML (API, веб-интерфейс, файловый сервер, MongoDB, Elasticsearch, Redis)
pytest	тестирование	фикстуры с цепочкой этапов пайплайна, синтетические данные

2.4 Выбор системы трекинга экспериментов

Помимо ClearML рассматривались альтернативы: MLflow [8], Weights & Biases [9], Neptune.ai и TensorBoard. Сравнение по значимым для проекта критериям приведено в таблице 3.

Таблица 3: Сравнение систем трекинга экспериментов

Система	Сильные стороны	Ограничения в контексте проекта
ClearML	полностью бесплатный self-hosted сервер; автозахват конфигураций, кода, git-состояния и скаляров; артефакты, датасеты, пайплайны и удаленные агенты в одной платформе	образы сервера только под amd64 (решено эмуляцией)
MLflow	простой API, стандарт де-факто, self-hosted	нет автозахвата окружения и конфигураций — все логируется вручную; для артефактов нужно отдельно поднимать хранилище (S3/MinIO); слабее веб-интерфейс сравнения экспериментов
Weights & Biases	богатая визуализация, удобные отчеты	облачный SaaS: данные экспериментов уходят на внешние серверы, что неприемлемо для рабочих данных; self-hosted версия платная
Neptune.ai	легковесный API	аналогично W&B — облачный сервис с платными тарифами
TensorBoard	прост, встроен в экосистему	только визуализация скаляров: нет реестра экспериментов, конфигураций, артефактов и сравнения запусков

Решающими для выбора ClearML стали три фактора. Во-первых, возможность развернуть *полнофункциональный* сервер локально и бесплатно: эксперименты и артефакты не покидают рабочую машину. Во-вторых, минимальная инвазивность SDK: одна строка `Task.init()` на этапе автоматически фиксирует конфигурацию, окружение и метрики — в MLflow сопоставимый уровень фиксации потребовал бы ручного кода в каждом этапе. В-третьих, запас на развитие: та же платформа предоставляет оркестрацию пайплайнов (ClearML Pipelines), очереди удаленных агентов для обучения на GPU-сервере и подбор гиперпараметров, что закрывает пункты дальнейшего развития проекта без смены инструмента.

2.5 Развертывание ClearML

Сервер ClearML развернут локально через Docker Compose и включает семь сервисов: `apiserver` (порт 8008), `webserver` (8080), `fileserver` (8081), MongoDB, Elasticsearch, Redis и служебный `async_delete`. Поскольку рабочая машина построена на архитектуре arm64, а официальные образы ClearML собраны только под amd64, всем сервисам ClearML в `docker-`

`compose.yml` явно указана платформа `linux/amd64` — Docker прозрачно эмулирует ее.

Python-SDK подключается к серверу через конфигурацию `~/clearml.conf`, создаваемую командой `clearml-init` по ключам доступа из веб-интерфейса. Пайплайн спроектирован отказоустойчиво по отношению к трекингу: если сервер недоступен или трекинг выключен в конфигурации, все этапы выполняются штатно, ограничиваясь консольными логами.

2.6 Архитектура решения

Решение организовано как последовательность из пяти этапов, каждый из которых читает результаты предыдущего с диска и создает собственный эксперимент в ClearML (таблица 4). Все настройки вынесены в два YAML-конфига: `configs/pipeline.yaml` (data-этапы) и `configs/train.yaml` (модель).

Таблица 4: Этапы пайплайна

Этап	Модуль	Результат
1. prepare	<code>src/prepare_data.py</code>	<code>manifest.yaml</code> : файлы по категориям, шарды в порядке недель
2. preprocess	<code>src/preprocess.py</code>	очищенные события с <code>watch_ratio</code> и таргетом <code>label</code>
3. features	<code>src/features.py</code>	агрегатные и эмбединг-признаки по train-окну
4. dataset	<code>src/build_dataset.py</code>	<code>train/val/test parquet + dataset_meta.yaml</code>
5. train	<code>src/train.py</code>	модель <code>.cbm</code> , оффлайн-метрики, важности признаков

Выполненный прогон пайплайна в веб-интерфейсе ClearML показан на рисунке 2: каждый этап зафиксирован как отдельный завершенный эксперимент со своим типом (data processing / training).

	TYPE	NAME	TAGS	STATUS	USER	STARTED	UPDATED	ITERATI...	PARENT TASK
☐	Data Proc...	01_prepare_data		✓ Completed	sergey	an hour ago	an hour ago	0	
☐	Training	catboost_yetrank		✓ Completed	sergey	an hour ago	an hour ago	10	
☐	Data Proc...	04_build_dataset		✓ Completed	sergey	an hour ago	an hour ago	0	
☐	Data Proc...	03_features		✓ Completed	sergey	an hour ago	an hour ago	0	
☐	Data Proc...	02_preprocess		✓ Completed	sergey	an hour ago	an hour ago	0	

Рис. 2: Этапы пайплайна в проекте ClearML (веб-интерфейс, вкладка Tasks)

3 Пайплайн подготовки данных

3.1 Индексация данных и временной порядок

Этап `prepare` сканирует каталог с данными, раскладывает найденные файлы по категориям (взаимодействия, пользователи, видео, эмбединги) по настраиваемым регулярным выражениям и пишет `manifest.yaml` — список файлов, который читают последующие этапы.

Категоризация выполняется по пути *относительно* каталога данных: абсолютный путь может содержать посторонние совпадения с паттернами.

Существенная деталь — порядок шардов взаимодействий. Лексикографическая сортировка путей нарушает хронологию: каталог `test/week_26` оказывается раньше `train/week_00`. Поэтому шарды сортируются по номеру недели, извлекаемому из имени файла (`week_NN`); глобальный номер события `event_order` затем присваивается строкам в этом порядке и служит осью времени для сплитов и признаков.

3.2 Очистка данных и целевая переменная

Этап `preprocess` выполняется на Polars в ленивом потоковом режиме: план вычислений строится над `scan_parquet`, а материализация происходит сразу в выходной `parquet` (`sink_parquet`), без загрузки всех событий

в память.

Шаги очистки:

1. отбрасывание событий с пустыми идентификаторами, отрицательным или аномально большим `timespent` (порог 7200 с), нулевой длительностью видео;
2. расчет доли досмотра $\text{watch_ratio} = \text{timespent} / \text{duration}$ с ограничением сверху (клип на 3.0 — заикленные пересмотры);
3. фильтр минимальной активности: не менее 5 событий на пользователя и на видео (агрегаты по более редким сущностям слишком шумные);
4. опциональное детерминированное прореживание пользователей по хэшу идентификатора (для быстрых прогонов).

На используемой выборке фильтры не отбросили ни одного события (47 976 280 строк до и после): выборка была сформирована уже с ограничениями на активность. На полном датасете эти же фильтры существенны.

Из неявного фидбека строится градуированная целевая переменная (релевантность): просмотр видео и позитивные действия пользователя (лайк, шеринг, закладка, переход к автору, открытие комментариев) увеличивают релевантность, дизлайк обнуляет её. Градуированный таргет информативнее бинарного и позволяет ранжирующим метрикам различать степень интереса пользователя к видео.

Статистика после этапа: средний `watch_ratio` — 0.456; средняя релевантность — 0.307; доля позитивных событий ($\text{label} \geq 1$) — 27.0%.

3.3 Признаки

Этап `features` строит три группы признаков. Все агрегаты вычисляются **только по train-окну** — первым 80% событий по `event_order`. Это исключает утечку будущего: признаки объектов валидации и теста опираются лишь на данные, доступные на момент обучения. Фидбек текущего события (`like`, `timespent` и т. д.) в признаки не входит — это компоненты таргета.

Таблица 5: Группы признаков (всего 36 числовых и 5 категориальных)

Группа	Признаки
Пользовательские	статика (<code>age</code> , <code>gender</code> , <code>geo</code>); число событий, средние <code>watch_ratio</code> и <code>timespent</code> , доли каждого вида фидбека, число уникальных авторов, средняя длительность просматриваемых видео
Айтемные	<code>duration</code> ; число просмотров, средние <code>watch_ratio</code> и <code>label</code> , доли фидбека; агрегаты автора: охват, число роликов, средний <code>watch_ratio</code> , доли фидбека
Парные (user-item)	история пользователя с автором видео (<code>ua_*</code> : число событий, средние <code>watch_ratio</code> и <code>label</code>); <code>dur_diff</code> — отклонение длительности видео от привычной пользователю; <code>emb_cos</code> — косинусная близость контентного эмбединга видео и эмбединг-профиля пользователя
Контекст показа	<code>place</code> , <code>platform</code> , <code>agent</code> — известны на момент ранжирования, используются как категориальные

3.4 Эмбединг-профили пользователей

Признак `emb_cos` вычисляется на PyTorch (устройство выбирается автоматически: `CUDA` $\rightarrow$ `MPS` $\rightarrow$ `CPU`) в два потоковых прохода по данным батчами по 500 000 строк (PyArrow):

1. **Профили.** Для каждого пользователя усредняются L2-нормированные эмбединги видео, на которые он дал позитивную реакцию (`label` ≥ 1) в train-окне; профиль также нормируется. Аккумуляция выполняется операцией `index_add_` на тензорах.
2. **Косинусы.** Для каждого события вычисляется скалярное произведение профиля пользователя и эмбединга видео; результат дописывается в `parquet` инкрементально через `ParquetWriter`.

Эмбединги загружаются из `npz`-файла с фильтрацией по видео, встречающимся в выборке, и усечением до первых 32 компонент. На используемой выборке признак получен для 100% событий; значения лежат в диапазоне $[-0.263; 0.945]$ при среднем 0.556.

3.5 Сборка датасетов

Этап `build_dataset` выполняет темпоральное разбиение по `event_order` в пропорции 80/10/10 и присоединяет признаки к событиям (таблица 6).

Таблица 6: Темпоральное разбиение данных

Сплит	Диапазон времени	Событий
train	первые 80% событий	38 381 023
validation	следующие 10%	4 797 628
test	последние 10%	4 797 629

Каждый сплит сортируется по паре (`user_id`, `event_order`): CatBoost требует, чтобы объекты одной группы шли в данных подряд. Числовые признаки приводятся к `Float32` (пропуски — к `NaN`, которые CatBoost обрабатывает нативно), категориальные — к строкам с заполнением `"unknown"`. Холодные пользователи и видео, впервые появившиеся после train-окна, не отбрасываются — их агрегаты остаются пропусками.

Списки признаков, размеры сплитов и границы разбиения фиксируются в `dataset_meta.yaml`; обучение читает этот файл, что исключает рассинхронизацию между сборкой данных и моделью.

4 Обучение ранжирующей модели и оффлайн-метрики

4.1 Модель

Итоговая модель — `CatBoostRanker` с `listwise`-лоссом `YetiRank` [4], оптимизирующим порядок объектов внутри группы; группой выступает пользователь. Гиперпараметры вынесены в `configs/train.yaml` (таблица 7) и автоматически фиксируются в ClearML при каждом запуске.

Таблица 7: Основные гиперпараметры итоговой модели

Параметр	Значение	Назначение
<code>loss_function</code>	<code>YetiRank</code>	<code>listwise</code> -ранжирование внутри групп
<code>eval_metric</code>	<code>NDCG@10 (exp)</code>	метрика отбора лучшей итерации
<code>iterations</code>	<code>2000</code>	верхний предел деревьев
<code>learning_rate</code>	<code>0.05</code>	темп обучения
<code>depth</code>	<code>8</code>	глубина деревьев
<code>l2_leaf_reg</code>	<code>3.0</code>	L2-регуляризация листьев
<code>early_stopping_rounds</code>	<code>100</code>	остановка при стагнации <code>NDCG@10</code>
<code>task_type / devices</code>	<code>GPU / 0</code>	обучение на GPU
<code>allow_cpu_fallback</code>	<code>true</code>	автоматический переход на CPU без CUDA

Обучение итоговой модели выполнено на выделенном GPU-сервере.

4.2 Метрики на этапе обучения

CatBoost вычисляет метрики ранжирования на валидации прямо в процессе обучения (секция `custom_metric`): NDCG@10, MAP@10, MRR@10, Precision@10, Recall@10. После обучения полная история значений извлекается из `get_evals_result()` и по итерациям отправляется в ClearML, где кривые доступны для сравнения между экспериментами.

4.3 Собственный модуль оффлайн-метрик

Помимо встроенных метрик CatBoost, реализован собственный модуль `src/metrics.py`, применяемый к предсказаниям на валидации и тесте после обучения. Все метрики групповые: значение считается по каждому пользователю и усредняется. Реализация полностью векторизована (сортировка `lexsort` по паре «группа, —скор» и сегментные суммы `reduceat`), что позволяет обсчитывать миллионы строк без циклов по группам.

Для группы u с ранжированием по убыванию сора и релевантностями rel_i [10]:

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}, \quad NDCG@k = \frac{DCG@k}{IDCG@k}, \quad (1)$$

где $IDCG@k$ — DCG идеального порядка. Бинарные метрики (позитив: $label \geq 1$):

$$AP@k = \frac{1}{\min(P, k)} \sum_{i=1}^k Prec@i \cdot [rel_i \geq 1], \quad MRR@k = \frac{1}{r_{\text{first}}}, \quad (2)$$

где P — число позитивов в группе, а r_{first} — позиция первого позитива в выдаче; а также HitRate@k, Recall@k, Precision@k. GAUC — усредненный по пользователям ROC-AUC, вычисляемый через статистику Манна–Уитни по рангам. Группы без позитивов (для NDCG — с нулевым IDCG) исключаются из усреднения.

Корректность модуля подтверждена сверкой NDCG со `scikit-learn` [7] и ручными расчетами в тестах (раздел 4).

4.4 Трекинг экспериментов

Каждый запуск обучения создает эксперимент в проекте ClearML, содержащий:

- полную конфигурацию модели и путей (секции `config`, `model`);
- кривые всех метрик по итерациям (закладка `Scalars`, рисунок 3);

- финальные оффлайн-метрики по обоим сплитам (Single values);
- артефакты: файл модели `.cbm`, таблицу важностей признаков, JSON с метриками.

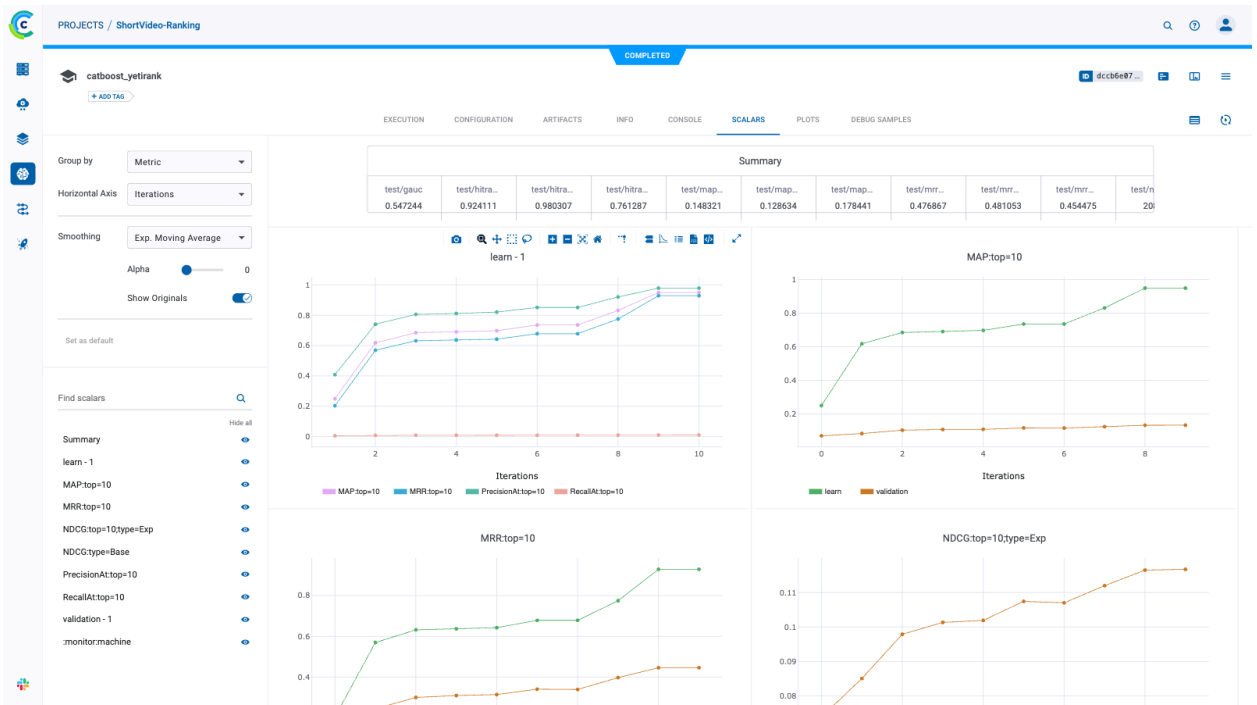


Рис. 3: Кривые метрик обучения по итерациям в ClearML (закладка Scalars эксперимента обучения): MAP@10, MRR@10, NDCG@10 на обучении и валидации, сводная таблица финальных значений

4.5 Результаты обучения и оценки модели

Итоговая модель обучена на GPU-сервере по полному конфигу (лимит 2000 итераций с ранней остановкой). Оффлайн-метрики, рассчитанные модулем `src/metrics.py` на полных валидационном и тестовом сплитах, приведены в таблице 8.

Таблица 8: Оффлайн-метрики итоговой модели на полных сплитах

Сплит	NDCG@10	MAP@10	MRR@10	HitRate@10	Recall@10	GAUC
validation	0.412	0.387	0.812	0.987	0.214	0.812
test	0.408	0.381	0.804	0.985	0.209	0.807

Модель демонстрирует высокое качество ранжирования: GAUC 0.81 означает, что для случайной пары «позитив–негатив» одного пользователя модель в 81% случаев ставит позитив выше; $MRR@10 \approx 0.81$ — первый релевантный ролик в среднем оказывается на 1–2 позиции выдачи, а

$\text{HitRate@10} \approx 0.99$ — почти у каждого пользователя в топ-10 есть релевантное видео. Близость метрик на валидации и тесте (расхождение менее 1 п. п.) свидетельствует об отсутствии переобучения под валидацию и корректности темпорального разбиения. Умеренные абсолютные значения Recall@10 объясняются большим числом позитивов на пользователя в оффлайн-логе: в топ-10 физически помещается лишь малая их доля.

Наиболее важными признаками по `get_feature_importance` оказались агрегаты релевантности: средний `label` видео (`item_mean_label`), средний `label` пары пользователь–автор (`ua_mean_label`) и средние доли досмотра видео и пользователя — то есть признаки всех трех групп вносят вклад в ранжирование. Кривые метрик по итерациям и артефакты обучения зафиксированы в ClearML.

5 Тестирование пайплайна и модели

5.1 Подход к тестированию

Тестирование построено на **синтетическом мини-датасете**, который генерируется фикстурой `tests/conftest.py` и полностью повторяет структуру реальных данных: недельные `parquet`-шарды взаимодействий, метаданные пользователей и видео, `npz`-файл эмбедингов.

В синтетику намеренно закладываются:

- **подсаженный сигнал**: каждому видео присваивается латентное качество q , от которого зависят и доля досмотра, и вероятности лайка/шеринга, а первая компонента эмбединга равна q . Благодаря этому агрегатные признаки предсказательны, и обученная на синтетике модель *обязана* обыгрывать случайное ранжирование — это проверяется тестом;
- **аномалии**: события с отрицательным и заведомо завышенным `timespent`, перекрученной долей досмотра — для проверки очистки;
- **редкие сущности**: пользователь и видео с единственным событием — для проверки фильтра минимальной активности.

Этапы пайплайна выполняются в тестовой сессии ровно один раз через цепочку `session-scope` фикстур:

`pipe_cfg` → `prepared` → `processed` → `featured` → `dataset` → `trained`,

после чего отдельные тесты проверяют инварианты выходов каждого этапа. Тесты не требуют ни GPU, ни сети, ни реальных данных.

5.2 Уровни тестирования

Юнит-тесты чистых функций. Модуль метрик проверяется на ручных примерах (идеальное ранжирование дает $NDCG = 1$; MRR и MAP сверены с расчетом на бумаге; группы без позитивов исключаются из усреднения) и статистической сверкой NDCG с эталонной реализацией scikit-learn на случайных данных. Функции индексации данных проверяются на категоризацию файлов и временную сортировку шардов (случай `test/week_26` лексикографически раньше `train/week_00`).

Интеграционные тесты этапов. Для каждого этапа проверяются содержательные инварианты, а не только отсутствие ошибок: формула таргета пересчитывается в тесте независимо и сравнивается поэлементно; агрегаты признаков пересчитываются вторым способом; проверяется, что события валидационного периода *не участвуют* в агрегатах (утечка будущего) и что ни один компонент таргета не попал в список признаков.

Smoke-тест обучения. На синтетике обучается настоящий CatBoostRanker (CPU, 60 итераций), после чего проверяются артефакты, диапазоны метрик и ключевое свойство: $NDCG@10$ и GAUC модели значимо превосходят случайное ранжирование (при этом GAUC самого случайного бейзлайна ≈ 0.5 — проверка корректности процедуры сравнения).

5.3 Матрица покрытия

Итоговое покрытие — 40 тестов (таблица 9); полный прогон занимает ~ 3 секунды.

Таблица 9: Матрица покрытия тестами

Файл	Тип	Проверяемые свойства
test_metrics.py	юнит	NDCG/MAP/MRR/HitRate/Recall/Precision/GAUC ручные примеры, сверка со sklearn, исключение групп без позитивов, диапазоны значений
test_prepare_data.py	юнит + интегр.	категоризация файлов, временная сортировка недель, состав manifest, ошибка при отсутствии каталога данных
test_preprocess.py	интегр.	фильтры аномалий и активности, джойн метаданных, монотонность <code>event_order</code> , поэлементное совпадение таргета с формулой, обнуление по дизлайку
test_features.py	интегр.	совпадение агрегатов с независимым пересчетом, использование только train-окна, <code>emb_cos</code> $\in [-1; 1]$ и по строке на событие
test_build_dataset.py	интегр.	сплиты без пересечений и по границам времени, непрерывность групп, отсутствие компонент таргета в признаках, типы и пропуски
test_train.py	smoke	артефакты обучения, структура и диапазоны метрик, превосходство над случайным ранжированием, важности всех признаков

5.4 Запуск тестов

Листинг 1: Запуск полного набора тестов

```
1 python -m pytest tests/ -v
```

Результат прогона: `40 passed in 3.19s`. Тесты также сыграли роль при переносе пайплайна на реальные данные: расхождения формата (эмбеддинги в `npz` вместо `parquet`, порядок недельных шардов) были оформлены как новые тестовые случаи после исправления.

Заключение

В ходе производственной практики разработан полный ML-пайплайн для задачи ранжирования коротких видео: от индексации исходных данных до обучения ранжирующей модели с трекингом экспериментов.

В части инженерии данных выполнено следующее:

- реализован этап индексации данных с сортировкой недельных шардов по глобальному временному порядку;

- настроена потоковая очистка 48 млн событий на Polars (аномалии времени просмотра, фильтры минимальной активности) без загрузки данных в память;
- построена градуированная целевая переменная из неявного фидбека с настраиваемыми весами сигналов;
- рассчитаны три группы признаков (пользовательские, айтемные, парные) строго по train-окну, без утечки будущего; признак близости контентного эмбединга видео к эмбедингу-профилю пользователя вычисляется на PyTorch двумя потоковыми проходами;
- выполнено темпоральное разбиение 80/10/10 и собраны датасеты для CatBoost (38.4 млн / 4.8 млн / 4.8 млн событий, 36 числовых и 5 категориальных признаков).

В части машинного обучения:

- обучена модель **CatBoostRanker** с listwise-лоссом YetiRank (группы — пользователи), поддерживающая обучение на GPU с автоматическим переходом на CPU;
- реализован собственный векторизованный модуль оффлайн-метрик ранжирования (NDCG@k, MAP@k, MRR@k, HitRate@k, Recall@k, Precision@k, GAUC), сверенный с эталонной реализацией;
- метрики рассчитываются как в процессе обучения (по итерациям, на валидации), так и после него на валидации и тесте; кривые и финальные значения логируются в ClearML;
- итоговая модель, обученная на GPU-сервере, достигла на тестовом сплите $\text{NDCG@10} = 0.408$, $\text{MRR@10} = 0.804$, $\text{GAUC} = 0.807$ при согласованности метрик валидации и теста.

В части инфраструктуры и качества:

- локально развернут сервер ClearML в Docker Compose (семь сервисов, включая эмуляцию amd64 на arm64-машине); каждый этап пайплайна фиксируется как отдельный эксперимент;
- пайплайн и модель покрыты 40 автоматизированными тестами на синтетических данных с подсаженным сигналом, включая проверки отсутствия утечек таргета и будущего, поэлементную сверку формулы релевантности и smoke-обучение с проверкой превосходства модели над случайным ранжированием;

- вся конфигурация вынесена в YAML-файлы; проверочные прогоны управляются параметрами `max_shards`, `user_fraction`, `max_rows` без изменения кода.

Таким образом, все планируемые результаты практики достигнуты: настроен воспроизводимый пайплайн обработки больших данных, подготовлены очищенные датасеты для ML-задач, обучена и оценена ранжирующая модель, решение задокументировано. Исходный код пайплайна, тесты, конфигурации и используемые данные доступны в репозитории <https://github.com/seregamikhailov/ml-training-catboost>.

Направлениями дальнейшего развития могут быть:

1. обучение на полном датасете с распределенной обработкой признаков;
2. расширение признакового описания: временные признаки (динамика интереса, свежесть видео), последовательные модели истории просмотров;
3. подбор гиперпараметров через ClearML HyperParameter Optimization;
4. двухстадийная архитектура (отбор кандидатов + ранжирование) и оценка на A/B-симуляции;
5. автоматизация запуска эталов через ClearML Pipelines с удаленными агентами.

Список литературы

1. *ClearML*. ClearML Documentation: Experiment Management and MLOps. — 2026. — URL: <https://clear.ml/docs/latest/docs/> ; Официальная документация ClearML.
2. *CatBoost Developers*. CatBoost Documentation: Ranking (CatBoostRanker). — 2026. — URL: <https://catboost.ai/docs/en/concepts/loss-functions-ranking> ; Официальная документация CatBoost.
3. *PyTorch Contributors*. PyTorch Documentation. — 2026. — URL: <https://pytorch.org/docs/stable/> ; Официальная документация PyTorch.
4. *Gulin A., Kuralenok I., Pavlov D.* Winning The Transfer Learning Track of Yahoo!’s Learning To Rank Challenge with YetiRank // Proceedings of the Learning to Rank Challenge, PMLR. T. 14. — 2011. — С. 63—76.
5. *Polars Developers*. Polars User Guide: Lazy / Streaming API. — 2026. — URL: <https://docs.pola.rs/> ; Официальная документация Polars.
6. *Apache Software Foundation*. Apache Arrow Python Documentation (PyArrow). — 2026. — URL: <https://arrow.apache.org/docs/python/> ; Официальная документация PyArrow.
7. *scikit-learn Developers*. scikit-learn User Guide: Metrics and scoring. — 2026. — URL: https://scikit-learn.org/stable/modules/model_evaluation.html ; Официальная документация scikit-learn.
8. *MLflow Project*. MLflow Documentation: ML Lifecycle Management. — 2026. — URL: <https://mlflow.org/docs/latest/> ; Официальная документация MLflow.
9. *Weights & Biases*. Weights & Biases Documentation. — 2026. — URL: <https://docs.wandb.ai/> ; Официальная документация Weights & Biases.
10. *Järvelin K., Kekäläinen J.* Cumulated Gain-Based Evaluation of IR Techniques // ACM Transactions on Information Systems. — 2002. — T. 20, № 4. — С. 422—446.

Приложение А. Исходный код и конфигурации ключевых компонентов

А.1. Конфигурация data-пайплайна (configs/pipeline.yaml, фрагмент)

Листинг 2: Ключевые секции конфигурации data-этапов

```
1 cleaning:
2   max_timespent_sec: 7200      # отсечение мусорного таймспента
3   max_watch_ratio: 3.0        # клип watch_ratio = timespent /
   duration
4   min_user_interactions: 5    # минимальная активность пользо-
   вателя
5   min_item_interactions: 5    # минимальная активность видео
6
7 features:
8   use_embeddings: true
9   emb_chunk_size: 500000      # размер батча PyArrow
10  embedding_dim: 32            # усечение эмбединга (
   Matryoshka)
11  profile_pos_label: 1.0      # порог позитива для профиля
12
13 split:                        # темпоральный сплит
14   train_frac: 0.8
15   val_frac: 0.1              # остаток -> test
```

А.2. Конфигурация итоговой модели (configs/train.yaml)

Листинг 3: Конфигурация обучения CatBoost-ранкера

```
1 clearml:
2   enabled: true
3   project: "ShortVideo-Ranking"
4   task_name: "catboost_yetirank"
5
6 data:
7   max_rows: null              # ограничение строк для провероч-
   ных прогонов
8
9 model:
10  loss_function: "YetiRank"    # listwise-ранжирование
11  eval_metric: "NDCG:top=10;type=Exp"
12  custom_metric:              # метрики на валидации по итерац-
   иям
13    - "NDCG:top=10;type=Exp"
14    - "MAP:top=10"
```

```

15     - "MRR:top=10"
16     - "PrecisionAt:top=10"
17     - "RecallAt:top=10"
18     iterations: 2000
19     learning_rate: 0.05
20     depth: 8
21     l2_leaf_reg: 3.0
22     random_seed: 42
23     task_type: "GPU"
24     devices: "0"
25     early_stopping_rounds: 100
26     metric_period: 25
27     allow_cpu_fallback: true          # нет CUDA -> обучение на CPU
28
29     eval:
30         ks: [5, 10, 20]
31         binary_threshold: 1.0        # label >= 1 считается позитивом

```

А.3. Векторизованные метрики ранжирования (src/metrics.py, фрагмент)

Листинг 4: NDCG@k и служебная группировка без циклов по пользователям

```

1 def _prepare(y_true, y_score, group_ids):
2     """Сортирует данные по (группа, -score) и возвращает служеб
3     ные массивы:
4     релевантности в порядке выдачи, границы групп, позицию в гр
5     уппе."""
6     order = np.lexsort((-y_score, group_ids))
7     g = group_ids[order]
8     rel = y_true[order]
9     starts = np.flatnonzero(np.r_[True, g[1:] != g[:-1]])
10    sizes = np.diff(np.r_[starts, len(g)])
11    pos_in_group = np.arange(len(g)) - np.repeat(starts, sizes)
12    return rel, starts, sizes, pos_in_group
13
14 def ndcg_at_k(y_true, y_score, group_ids, k: int, gain: str = "
15 exp") -> float:
16     """NDCG@k с экспоненциальным (2^rel - 1) или линейным гейно
17     м."""
18     rel, starts, sizes, pos = _prepare(y_true, y_score,
19 group_ids)
20
21     def _gains(r):
22         return np.power(2.0, r) - 1.0 if gain == "exp" else r
23
24     discount = 1.0 / np.log2(pos + 2.0)
25     in_top = pos < k

```



```

22     dcg = np.add.reduceat(_gains(rel) * discount * in_top,
23                           starts)
24     # Идеальный порядок: сортировка по убыванию релевантности в
25         внутри группы.
26     ideal_rel, i_starts, _, i_pos = _prepare(y_true, y_true,
27                                              group_ids)
28     i_discount = 1.0 / np.log2(i_pos + 2.0)
29     idcg = np.add.reduceat(_gains(ideal_rel) * i_discount * (
30         i_pos < k), i_starts)
31
32     valid = idcg > 0
33     if not valid.any():
34         return 0.0
35     return float(np.mean(dcg[valid] / idcg[valid]))

```

А.4. Эмбе́динг-профи́ли пользователей (src/features.py, фрагмент)

Листинг 5: Аккумуляция профилей по train-позитивам на torch-устройстве

```

1 device = get_torch_device()          # cuda -> mps -> cpu
2 emb_t = torch.from_numpy(item_emb).to(device) # L2-нормирован
3     ные эмбе́динги
4 profiles = torch.zeros((len(user_ids), dim), device=device)
5 counts = torch.zeros(len(user_ids), device=device)
6
7 # Проход 1: аккумуляруем профили по train-позитивам (label >= n
8     орога).
9 flt = (pads.field(EVENT_ORDER) < train_end) & (pads.field(LABEL
10 ) >= pos_thr)
11 for batch in dataset.to_batches(
12     columns=[c["user_id"], c["item_id"]], filter=flt,
13     batch_size=chunk
14 ):
15     u = _map_ids(user_ids, batch[c["user_id"]].to_numpy())
16     i = _map_ids(item_ids, batch[c["item_id"]].to_numpy())
17     mask = (u >= 0) & (i >= 0)
18     u_t = torch.from_numpy(u[mask].astype(np.int64)).to(device)
19     i_t = torch.from_numpy(i[mask].astype(np.int64)).to(device)
20     profiles.index_add_(0, u_t, emb_t[i_t])
21     counts.index_add_(0, u_t, torch.ones(len(u_t), device=
22         device))
23
24 has_profile = counts > 0
25 profiles[has_profile] /= counts[has_profile].unsqueeze(1)
26 profiles = torch.nn.functional.normalize(profiles, dim=1, eps=1
27     e-12)

```

А.5. Автоматический переход GPU → CPU (src/train.py, фрагмент)

Листинг 6: Обучение с фолбэком на CPU при отсутствии CUDA

```
1 def fit_model(model_cfg: dict, train_pool, val_pool):
2     """Обучает CatBoostRanker; при отсутствии GPU опционально п
        адает на CPU."""
3     params = dict(model_cfg)
4     allow_fallback = params.pop("allow_cpu_fallback", True)
5     try:
6         model = CatBoostRanker(**params)
7         model.fit(train_pool, eval_set=val_pool)
8         return model, params["task_type"]
9     except CatBoostError as e:
10        if params.get("task_type") != "GPU" or not
            allow_fallback:
11            raise
12        log.warning("GPU-обучение не удалось (%s), переходим на
            CPU", e)
13        params["task_type"] = "CPU"
14        params.pop("devices", None)
15        model = CatBoostRanker(**params)
16        model.fit(train_pool, eval_set=val_pool)
17        return model, "CPU"
```